

Project Directory Structure

2023-10-10

SOP STATUS: Completed ▾ **REVIEW STATUS:** Approved ▾

AUTHORS

Bo Meyering Spencer Barriball Brandon Schlautman

PURPOSE

The purpose of this document is to standardize the directory structure of a new project for use within the perennial legumes program at TLI. The specific details covered are the project directory structure with all folders, intent of use for each folder, required and optional files, and specifications for those file types.

SCOPE

This procedure is limited to the scope of creating new project directories, which are used to track data that is collected in the project, methods of the project design and implementation, and a README file for others to refer to in the future. The scope of this procedure excludes other types of directories for specific and limited purposes such as software development, machine learning projects, publications, etc. This procedure is further limited to use within the perennial legumes program and may not apply to other research projects within the larger TLI or research community.

REFERENCES AND RELATED DOCUMENTS

- Working with [Git](#) and [Github](#)
- Git [official documentation](#)
- Markdown syntax [documentation](#)

MATERIALS

- A computer with access to the internet.

DEFINITIONS

- src: Short for source. This folder normally contains source code in a software development project.
- README: An entypoint document for people who want to know about the structure of the project.
- CLI: Command line interface. Terminal on Linux and Mac OS. Command Prompt, Windows Powershell, or Git Bash for Windows.
- Git: A CLI program for handling version control of files and directory structure.

DETAILED PROCEDURE

Cloning the Github Repository

1. Ensure that you have Git installed locally on your computer, or that you can easily get to Github from a browser.
 - a. Git can easily be installed on all operating systems by downloading the installer or source binaries from <https://git-scm.com/downloads>. Most Mac and Linux distributions come with Git by default.
2. Clone the template directory from [here](#) by either cloning the repository using your CLI or by downloading the git repository directly and unzipping it in the desired location of your choice.
 - a. Cloning from Git CLI (Windows)
 - i. If you are on a Windows operating system, open the main folder where you want to start a new project. So, if you have a folder in Google drive called 'projects' in which you store all your project folders, navigate to that folder.
 - ii. Right click to open the context menu and open 'Git Bash Here'.

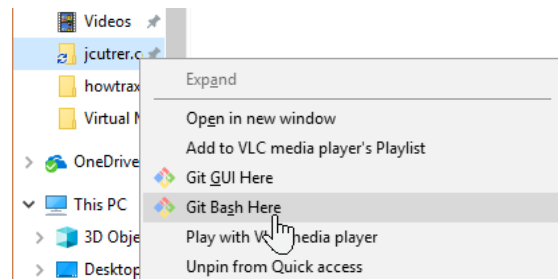


Figure 1. Right click context menu for Git Bash in Windows

- iii. Inside the Git bash shell, clone the directory with the following command:

Unset

```
git clone https://github.com/TLILegume/project\_init\_template.git
```

- iv. Navigate to the new directory.
- b. Cloning from Git CLI:
 - i. On a Unix based operating system, navigate to the projects folder and right click to open the context menu and select 'Open in terminal' or 'New terminal at folder', depending on the operating system.
 - ii. Open a terminal window and paste the following command:

Unset

```
git clone https://github.com/TLILegume/project\_init\_template.git
```

- iii. Navigate to the new directory.

- 3. Now you should have a folder structure in the projects folder like this:

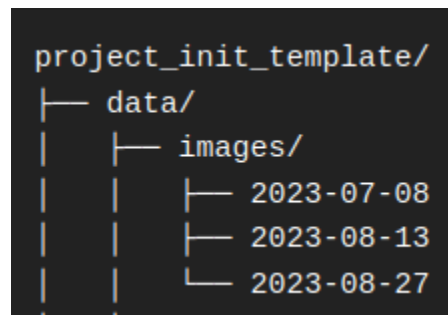
'/projects/project_init_template'

```
project_init_template/  
├── data/  
│   ├── images  
│   ├── raw  
│   └── processed  
├── doc  
├── output  
├── src  
├── README.md  
└── .git
```

Figure 2. Template project directory structure

Defining the Subfolders

1. The root folder 'project_init_template' should be renamed according to the appropriate project naming conventions.
2. data: This folder is the parent folder for all data in the project. It has to contain the following three subfolders:
 - a. images: This folder is used to store images relating to the project. It can be subdivided into other folders/hierarchies as necessary, for instance, by date.



```
project_init_template/  
├── data/  
│   ├── images/  
│   │   ├── 2023-07-08  
│   │   ├── 2023-08-13  
│   │   └── 2023-08-27
```

Figure 3. Dated folders for image storage in project_init_template/data/images

- b. raw: This folder contains all of the raw data used in the project, i.e. csv, text, or Excel files that are directly downloaded from the internet, lab equipment, or field machinery/instruments; The data in this folder are **not** ready to be used or analyzed, but are data that require cleaning, reformatting, or filtering.
 - c. processed: This folder contains all of the processed data used in the project, i.e. files that have been cleaned (missing values removed or imputed, incorrect values scrubbed or corrected, subset of data needed for analysis, aggregated data, etc.). If the data you collect is already processed, i.e. if it needs no cleaning, you may place it here directly.
 - d. doc: This folder contains all documentation for the project, including, but not limited to, project experimental design, project narratives and scope, project field maps, metadata, and any other documentation needed for users to orient themselves with purpose and scope of the project. The documents used here should be formatted as
 - i. Google Docs (.gddoc)
 - ii. Word 365 (.docx)
 - iii. Google Sheets (.gdsheet)
 - iv. Excel 365 (.xlsx)
 - v. CSV files (.csv)

- vi. Markdown files (.md)
- vii. Plain text files (.txt)
- e. output: This folder contains all the **non-data** output from a project. These could be images that were processed and saved for a publication figure, graphs, summary documents of data analysis, PDF files, etc. Anything that can genuinely be considered output of the project may reside here.
- f. src: The source folder should contain only code and code documents (.Rmd, .ipynb, etc.) for any preprocessing, analysis, and modeling done in the project. The exact sub-structure of this folder greatly depends on the level of coding needed for the project, and is left to the discretion of the project manager, but could look something like this:

```
├── src/
│   ├── analysis/
│   │   ├── my_great_project.Rmd
│   │   ├── .Rdata
│   │   └── .Rhistory
│   ├── utils/
│   │   └── image_utils.py
│   ├── modeling/
│   │   ├── image_processing.py
│   │   └── prediction.py
│   └── main.py
├── README.md
└── .git
```

Figure 4. An example of code file structure for project_init_template/src

- g. test: This folder is optional for non-coding related projects, but it should contain all of the unit tests for particular code classes or scripts. This is a mirror approach to unit testing, and the structure might look something like this

```
├── test/
│   ├── utils/
│   │   └── image_utils_test.py
│   ├── modeling/
│   │   ├── image_processing_test.py
│   │   └── prediction_test.py
├── README.md
└── .git
```

Figure 5. Example of folder structure for project_init_template/test

- h. README.md: This file is a generic README file that should be **edited** and **customized** for your project. This is the entrypoint for anyone to know what the main goal of the project is, and where to find certain information regarding the data, output, or source code. **Do** fill it out, it does not have to be long. README files are written in Markdown, learn more about Markdown formatting [here](#).
- i. .git: This is a hidden folder by default. You can use it if you plan to use Git and Github for version control (*which you should!*).

Usage

1. Start using the new project directory by changing/customizing the README files, adding documentation, adding data, etc.
2. Keep the directory under version control using Git.

DATA STORAGE INSTRUCTIONS

1. The new project directory should be backed up to the shared TLI_Legume drive when started so that lab members working on the project have access to the project data and documentation.

FLOWCHART

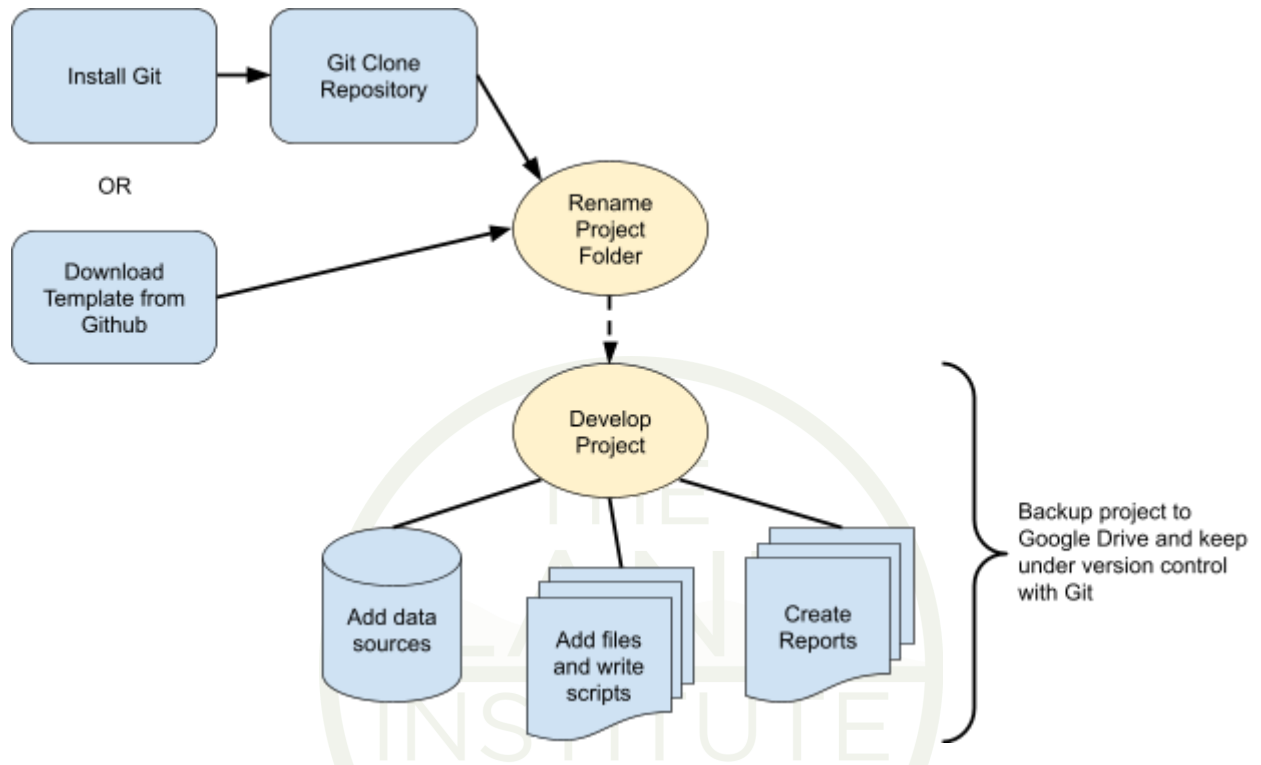


Figure 6. Overall process of creating and using a new project directory

TROUBLESHOOTING

- Contact the perennial legumes program [data manager](#) if you have any questions.

NOTES

Git and Github

Using Git and Github, even at a rudimentary level, is an extremely important step of proper data and code management for aspiring developers, data scientists, and just data savvy people. There are many excellent tutorials online that cover the basics as well as more advanced topics.

